

Orcaella: Hybrid Fault Tolerance with Client-Selectable Finality Latency

Abstract—Classical partially synchronous state machine replication, as in PBFT [1], tolerates f Byzantine replicas among $n \geq 3f + 1$ using three communication steps per request. Recent protocols such as Minimmit [2] achieve *two-message-delay* decisions under stronger size assumptions, notably $n \geq 5f + 1$ when any silent replica must be counted as a potential equivocator. Hydrangea [3] and Kudzu [4] treat mixed Byzantine and crash faults, focusing on providing a fast-path under optimistic conditions while maintaining a fall-back commitment path similar to PBFT. In this paper, we also consider a mixed model, but focus on studying the fault tolerance of the 2-message-delay commit. For this, we prove a tight bound of $n \geq 5f + 3c + 1$. Extending this result, we also show that there exists a more resilient commit path that allows an extra $f_{abc} < n - 3f - 2c$ alive-but-corrupt [5] faults at 4-message-delays. Core liveness is claimed in executions with at most f equivocators; if this regime is violated (e.g., AbC-induced forks), the protocol enters synchronous recovery, where only the resilient-path safety guarantee is preserved. As a result, for $f = 16$, $c = 6$, and $n = 99$, we obtain a commit path that tolerates 22% of replicas failing for liveness, 16% equivocating for 1-RTT safety, and 54% equivocating for 2-RTT safety, where one RTT equals two message delays for clients collocated with replicas.

I. INTRODUCTION

State machine replication (SMR) protocols are the foundational building blocks of modern decentralized systems [6]. Under partial synchrony [7], classical solutions like PBFT [1] tolerate up to f Byzantine replicas among $n \geq 3f + 1$, but require three all-to-all communication delays to reach consensus. As decentralized applications demand ever-lower latency, recent research has introduced *two-message-delay* protocols (e.g., Minimmit [2]). However, achieving this optimal fast-path latency traditionally requires a much larger committee size of $n \geq 5f + 1$, as every silent replica must be conservatively treated as a potential Byzantine equivocator.

The challenge. To mitigate the severe $5f + 1$ requirement, a natural approach is to distinguish between fully Byzantine faults (who may equivocate) and crash faults (who merely go silent). Recent work like Hydrangea [3] and Kudzu [4] adopt this mixed fault model. However, existing hybrid systems focus on providing a fast-path under optimistic conditions, while eagerly falling back to a slower, PBFT-style 3-round path when faults increase.

This leaves an open challenge: *How can we maximize the fault tolerance of the optimal 2-message-delay commit itself*

when relaxing the fault model? Furthermore, if a client does not care about this lower latency, *can we enhance the mixed fault model to provide even stronger, FlexibleBFT-style [5] safety guarantees?*

Our solution. In this paper, we answer these questions by formally defining the tight quorum intersections required to maximize 2-message-delay commits under separate Byzantine (f) and crash-faulty (c) caps. We show that such a system necessitates:

$$n \geq 5f + 3c + 1$$

with an optimal fast-path quorum of $q = n - f - c$, i.e., $4f + 2c + 1$ at the minimal committee size. At $c = 0$, this recovers the familiar $5f + 1$ regime [8], [2], but when crashes are separated, it allows for configurations with better liveness guarantees.

Building on this foundational result, we introduce Orcaella, a hybrid protocol that exposes two explicit finality paths for clients. These paths are built using *Quorum Certificates* (QCs)—cryptographic proofs consisting of q matching signed messages from distinct replicas:

- **Optimal Fast-Path (2-delay):** Clients can finalize quickly after collecting a single quorum of votes (VoteQC), providing safety against f Byzantine faults.
- **Resilient Path (4-delay):** Clients willing to wait for two additional rounds (chaining a CheckpointQC and FinalityQC) gain resilience against an additional f_{abc} *alive-but-corrupt* [5] replicas. These are replicas that participate but may equivocate.

Crucially, the resilient path provides safety when the core execution assumptions are violated and forks occur. In executions with at most f equivocators, the protocol retains its normal liveness behavior. If this regime is violated (e.g., AbC-induced forks), liveness is temporarily lost, replicas enter synchronous recovery, and resilient-path safety remains preserved throughout.

Concrete trade-offs. This dual-path architecture allows operators to explicitly navigate the latency-safety-liveness trade-off. Depending on the expected environment, for a deployment of $n \approx 100$ replicas, an operator can tune their configuration (Figure 1):

- **Balanced ($f = 12, c = 13$):** Tolerates up to 25 offline replicas for liveness, ensures 1-RTT (two-message-delay) safety against 12 Byzantine faults, and 2-RTT safety

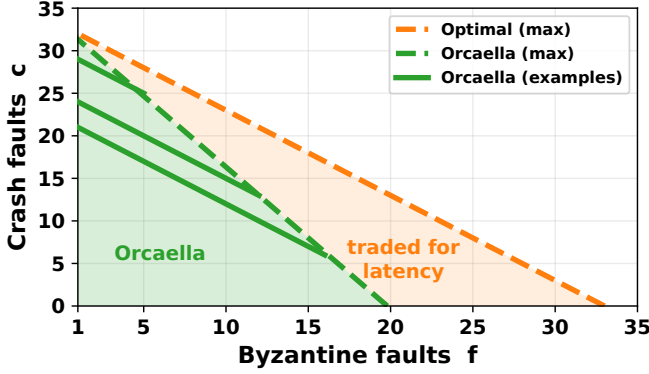


Fig. 1: Fault-tolerance design space at $n = 100$. Typical $3f + 1$ protocols treat every fault as Byzantine, covering the region $f + c \leq 33$ (orange) at the cost of requiring 3 message delays to commit. Orcaella instead covers the region $5f + 3c + 1 \leq n$ (green), trading some fault tolerance for its 2-message-delay commit. Each solid green line is one example config’s runtime frontier: a cap (f, c) tolerates any mix down to $(0, f + c)$.

against a maximum of 49 total equivocators (strictly below the intersection bound $T = 50$).

- **Byzantine-heavy** ($f = 16, c = 6$): Optimizing for more malicious conditions slightly reduces liveness tolerance (22 offline replicas) but boosts 1-RTT safety to 16 and 2-RTT safety to a maximum of 54 total equivocators ($< T = 55$).
- **Crash-heavy** ($f = 5, c = 25$): Assuming a more benign but flaky network maximizes liveness (tolerating 30 offline replicas) while still offering 2-RTT safety against a maximum of 40 total equivocators ($< T = 41$).

This flexibility allows deployments to adapt to their exact threat models without sacrificing the optimal 2-message-delay commit. Although for the scope of this paper and all our results we consider $f > 0$ we note that Orcaella can actually be configured with $f = 0$ and $n = 2c + 1$ to provide a CFT variant without any code changes other than configuring the quorums. Appendix D shows an evaluation of this variant for completeness.

Contributions. We make the following contributions:

- 1) The tight quorum inequalities ($n \geq 5f + 3c + 1$) that maximize the fault tolerance of 2-delay BFT consensus (Theorem 1).
- 2) Orcaella, a protocol that exposes the 2-delay and 4-delay paths, including a fully specified view change and fork recovery mechanism.
- 3) FlexibleBFT-style client analyses formalizing the safety vs. liveness trade-offs and proving the 4-delay path remains safe against an additional $f_{abc} < n - 3f - 2c$ alive-but-corrupt faults.
- 4) OrcDAG, a DAG-based instantiation of Orcaella over an uncertified-DAG fabric, and its evaluation on realistic deployment quantifying the latency gained by trading fault tolerance.

TABLE I: Notation

Symbol	Meaning
n	Total replicas
f	Byzantine (equivocating) cap
c	Crash-faulty cap
f_{abc}	Alive-but-Corrupt cap (Resilient Path client analysis)
q	Quorum threshold, $q = n - f - c$
k	View-Change accept threshold, $k = 2f + c + 1$
σ	State root after applying a slot ($\sigma_s = H(\sigma_{s-1} h)$)
Δ	Post-GST bound on message delay

II. NETWORK AND FAULT MODEL

We use standard message-passing state machine replication [9] among n replicas (the *leader* or *primary* proposes each slot) under partial synchrony [7]. Channels are authenticated; digital signatures identify senders. We distinguish four replica *behaviors*:

- **Byzantine** replicas: may crash or send arbitrary messages including *equivocations* (inconsistent signed statements to different recipients) on any protocol message;
- **Crash-faulty** replicas: follow the protocol while active and may then permanently stop sending messages without equivocating;
- **Alive-but-Corrupt** (AbC) replicas [5]: remain live and participate in the protocol but may equivocate on any message they send;
- **Correct** replicas: follow the protocol and remain live throughout.

Rather than fixing a single static fault partition, Orcaella states each guarantee over one of two *nested* families of executions:

- **Core executions**, in which at most f replicas equivocate (Byzantine) and at most c crash, leaving at least $n - f - c$ correct replicas. Every core execution enjoys Fast-Path safety *and* liveness; all liveness analysis in this paper assumes this family, i.e., at most f equivocators.
- **Extended executions**, in which at most $f + f_{abc}$ replicas equivocate (f Byzantine plus $f_{abc} < n - 3f - 2c$ AbC) and at most c crash, leaving at least $n - f - c - f_{abc}$ correct replicas. In this strictly larger family only Resilient-Path safety is guaranteed; liveness may be temporarily lost until synchronous recovery (Algorithm 2).

The same protocol runs unmodified in both families; each client chooses which finality rule to rely on—VoteQC (Fast Path) or FinalityQC (Resilient Path)—and thereby which family’s guarantee it obtains.

The standard partial synchrony network model [7] assumes that after an unknown Global Stabilization Time (GST), message delays between correct replicas are bounded by a known constant Δ . The adversary controls all non-honest replicas up to the misbehavior they are allowed to perform and the network scheduling (subject to this bound after GST). It cannot break cryptography.

III. THE ORCAELLA PROTOCOL

This section instantiates the fault model and Fast-Path/Resilient Path split from Section II into a message-level protocol. It uses the thresholds derived in Theorem 1: $n \geq 5f + 3c + 1$ and $q = n - f - c$ (equal to $4f + 2c + 1$ at the minimal n). Replicas communicate on authenticated channels under partial synchrony. The protocol proceeds in views $v \in \mathbb{N}$, each with a designated leader $\text{LEADER}(v)$, and assigns proposals to sequence numbers $s \in \mathbb{N}$.

A. The protocol

Algorithm 1 specifies the protocol, covering both normal operation and leader recovery. Normal operation is divided into a Fast-Path (Steps 0–1, lines 2 to 4) that guarantees safety against f Byzantine faults in two message delays, and a Resilient Path (Steps 2–4, lines 8 to 14) that provides extended safety against alive-but-corrupt (AbC) faults at the cost of two additional message delays.

Client finalization paths. Clients finalize transactions by observing the certificates produced by the protocol. The latency bounds stated in this paper (e.g., two or four message delays) assume that clients are colocated on the replicas participating in the consensus protocol. If clients are external, their perceived finality latency naturally increases by the round-trip time (RTT) to communicate with the replica set, as is standard in all BFT protocols.

- **Fast-Path (2-delay):** A client finalizes (v, s, h) immediately upon collecting a **VoteQC**. This provides optimal latency and safety against f Byzantine faults.
- **Resilient Path (4-delay):** A client finalizes (s, h, σ) upon obtaining a **FinalityQC** (output by a replica in Step 4, line 14) together with its underlying **CheckpointQC**. This trades higher latency for extended safety against an additional f_{abc} alive-but-corrupt faults, without interfering with the core protocol’s liveness [5].

Protocol description. The leader of view v proposes a batch of transactions. Replicas validate the proposal and cast an **accept** vote, but only if the proposal is consistent with the branch $\mathcal{O}$ adopted from the latest **NEWVIEW** (line 4); this guard is what carries Fast-Path commits across view changes (Lemma 4). Correct replicas send at most one vote per slot (v, s) . If a client gathers q such **accept** votes, it forms a **VoteQC** and can immediately safely execute the payload (Fast-Path). If the leader is faulty [10] or the network is asynchronous, replicas eventually timeout and broadcast **VIEWCHANGE** messages, which collectively serve as an implicit abandon for uncommitted slots.

To support the Resilient Path, replicas that observe a **VoteQC** compute the resulting deterministic state root σ and broadcast a checkpoint proposal. Each **CHKPROP** carries its justifying **VoteQC**, so conflicting checkpoint proposals surface conflicting **VoteQCs** as fork evidence. We assume state roots bind the execution history: σ_s commits to the previous root and the applied digest (e.g.,

$\sigma_s = H(\sigma_{s-1} \parallel h)$), so a root at height s determines the checkpointed content at every height below it. **Safety is strictly guarded by Step 2** (line 8): to prevent conflicting checkpoints, an honest replica broadcasts at most one **CHKPROP** per slot height. For this reason, checkpoint artifacts deliberately carry no view number: a slot’s **VoteQC** may form in view v at one replica and in a later view v' at another (with the same digest, by Fast-Path safety), and view-tagged **CHKPROPs** would split each replica’s single permitted proposal across views, permanently blocking the checkpoint. While this strict locking rule means *liveness* can be lost if honest replicas split their **CHKPROP** messages across different branches (a trade-off we explicitly accept), it provides an ironclad safety guarantee. Once q replicas agree on this checkpoint (forming a **CheckpointQC**), they lock it by broadcasting a **CHKWITNESS** message. Note that **CHKWITNESS** messages attest to the checkpoint *content* (s, h, σ) rather than to one specific certificate: two **CheckpointQCs** over the same content but different signer sets may circulate, and binding witnesses to a particular certificate would needlessly split them. To safely support a checkpoint, an honest replica will broadcast a **CHKWITNESS** as long as the checkpoint’s state σ matches its own deterministic execution of the payload ($\sigma = \text{APPLY}(h)$), even if its single permitted **CHKPROP** for that height was already spent on a different, unfinalized branch. Once q such witness messages are collected, a replica outputs a **FinalityQC** (Step 4, line 14) as its Resilient Path finality decision for that slot.

Notice that with f_{abc} alive-but-corrupt faults, there can be **VoteQC** forks across different views. However, because honest replicas only broadcast a single **CHKPROP** per height, the double-signing bound (Lemma 7) guarantees that at most one valid **CheckpointQC** can ever be formed per height. Any resulting loss of liveness on the Resilient Path is cleanly resolved by the Fork Recovery protocol (Algorithm 2) during a synchronous epoch. Steps 2–4 (lines 8 to 14) ensure that this single **CheckpointQC** is witnessed by the network and the resulting **FinalityQC** is propagated before resilient finalization. Notice that every protocol artifact (**VoteQC**, **CheckpointQC**, **FinalityQC**) uses the exact same quorum threshold q .

View-Change details. The view change protocol is structurally similar to both PBFT and modern protocols like Streamlet [11] or HotStuff [12], [13]. If $\text{LEADER}(v + 1)$ fails to assemble a valid **NEWVIEW** before a timeout, replicas re-issue **VIEWCHANGE** messages for view $v + 2$ with doubled timeouts, following standard view synchronization [1]; a replica that observes a valid **VIEWCHANGE** or **NEWVIEW** for a view higher than its own joins it immediately, broadcasting its own **VIEWCHANGE** for that view. Assuming the core limit of f Byzantine faults holds (i.e., there are no **VoteQC** forks), any two sets of q **VIEWCHANGE** messages intersect at an honest replica ($2q - n > f$). This guarantees that any **VoteQC** used by

Algorithm 1: Orcaella: Normal Operation & Leader Recovery for replica r

```

1: Definitions & State Variables:
   Thresholds:  $q = n - f - c$  and  $k = 2f + c + 1$ .
   VoteQC:  $q$  matching VOTE messages for  $(v, s, h)$  with accept.
   CheckpointQC:  $q$  matching CHKPROP messages for  $(s, h, \sigma)$ .
   FinalityQC:  $q$  matching CHKWITNESS messages for  $(s, h, \sigma)$ .
    $\text{current\_view} \leftarrow 0$ 
    $\mathcal{O} \leftarrow \emptyset$  ▷ Branch adopted from the latest NEWVIEW ( $\perp$  = unconstrained)

2: (step 0) upon  $r = \text{LEADER}(v)$  and  $v = \text{current\_view}$  and new payload  $B_{v,s}$  do
3:   broadcast  $\langle \text{PROPOSE}, v, s, B \rangle_r$ 

4: (step 1) upon receiving  $\langle \text{PROPOSE}, v, s, B \rangle_L$  from  $L = \text{LEADER}(v)$  do
5:   if  $v = \text{current\_view}$  and  $r$  has not yet voted for slot  $s$  in view  $v$  then
6:     if  $\mathcal{O}[s] \in \{\perp, H(B)\}$  then ▷ Vote only consistent with the adopted branch
7:       broadcast  $\langle \text{VOTE}, v, s, H(B), \text{accept} \rangle_r$ 

8: (step 2) upon observing a valid VoteQC  $Q$  for  $(v, s, h)$  do
9:   if no CHKPROP has been broadcast for slot  $s$  then
10:     $\sigma_s \leftarrow \text{APPLY}(h)$  ▷ Deterministic state transition
11:    broadcast  $\langle \text{CHKPROP}, s, h, \sigma_s, Q \rangle_r$  ▷  $Q$  justifies the proposal; excluded from quorum matching

12: (step 3) upon observing a CheckpointQC  $C$  for  $(s, h, \sigma)$  where  $\sigma = \text{APPLY}(h)$  do
13:   broadcast  $\langle \text{CHKWITNESS}, s, h, \sigma \rangle_r$ 

14: (step 4) upon collecting  $q$  matching  $\langle \text{CHKWITNESS}, s, h, \sigma \rangle$  messages do
15:   output  $\langle \text{FINALITYQC}, s, h, \sigma \rangle$  ▷ Resilient Path finality decision for slot  $s$ 

16: upon timeout in view  $v$  or timeout awaiting  $\langle \text{NEWVIEW}, v \rangle$  do ▷ Timeouts double per attempt
17:    $\text{current\_view} \leftarrow \perp$  ▷ Halt normal operation
18:    $\mathcal{P}^r \leftarrow \{\text{highest-view } \langle \text{VOTE}, v', s, h, \text{accept} \rangle \text{ cast by } r \text{ for each } s\}$ 
19:   broadcast  $\langle \text{VIEWCHANGE}, v + 1, \mathcal{P}^r \rangle_r$ 

20: upon  $r = \text{LEADER}(v)$  and receiving  $q$  valid  $\langle \text{VIEWCHANGE}, v, \mathcal{P}_i \rangle$  messages  $\mathcal{V}$  do
21:   for each slot  $s$  represented in  $\mathcal{V}$  do
22:      $\mathcal{K} \leftarrow \{h \mid h \text{ is the highest-view vote for } s \text{ in at least } k \text{ reports in } \mathcal{V}\}$  ▷ Aggregated per digest, across views
23:     if  $\mathcal{K} \neq \emptyset$  then
24:        $\mathcal{O}[s] \leftarrow \text{digest in } \mathcal{K} \text{ with the highest reported view}$  ▷ Singleton if slot  $s$  has a VoteQC (Lemma 4); ties broken by smallest digest
25:     else
26:        $\mathcal{O}[s] \leftarrow \perp$  ▷ No locked value; safe to propose arbitrary valid block
27:     broadcast  $\langle \text{NEWVIEW}, v, \mathcal{V}, \mathcal{O} \rangle_r$ 

28: upon receiving valid  $\langle \text{NEWVIEW}, v, \mathcal{V}, \mathcal{O} \rangle_L$  from  $L = \text{LEADER}(v)$  do
29:   verify  $\mathcal{O}$  against  $\mathcal{V}$  ▷ Deterministically recompute  $\mathcal{K}$  and  $\mathcal{O}$ 
30:    $\text{current\_view} \leftarrow v$  ▷ Enter new view
31:   adopt branch  $\mathcal{O}$  and resume normal operation

```

a client to finalize a Fast-Path commit is reported to the new leader and preserved in the new view's starting state $\mathcal{O}$. The new leader may hold only the digest h for a re-proposed slot; it fetches the block body from the $\geq q - f$ honest replicas that voted for h , since honest replicas vote only on proposals they store. Since replicas may re-vote for the same digest across views, the leader counts the reported votes per digest rather than per view (Lemma 4). By operating over a large $5f + 1$ style quorum, the protocol acts as a generalized 2-chain Streamlet, enabling rapid 2-message-delay finality instead of the standard 3-chain required under $3f + 1$. If the f limit is exceeded (e.g., by AbC faults double-voting) and a VoteQC fork occurs, Fast-Path safety is broken and preserving it is no longer required; instead, replicas halt and rely on Fork Recovery to salvage the Resilient Path. Note that for simplicity, Algorithm 1 requires replicas to send all their highest-view votes. In a practical implementation, replicas would use CheckpointQCs to prove a globally agreed upon state and safely truncate the required vote history, significantly reducing the size of VIEWCHANGE messages [14].

B. Fork recovery

Algorithm 2 specifies the procedure for detecting and recovering from forks (when the core f Byzantine limit is exceeded but clients rely on Resilient Path safety).

Synchronous network model. Unlike normal operation which relies on partial synchrony, this recovery procedure operates under a strictly synchronous network model. We assume that message delays between correct replicas are guaranteed to be bounded by a known, pessimistic constant Δ_{sync} . Replicas anchor the epoch's lockstep round structure at the delivery of the first valid ALARM (which every correct replica re-broadcasts), so the round boundaries of correct replicas are offset by at most Δ_{sync} —a bounded skew that synchronous broadcast protocols tolerate by standard techniques [15].

Fork recovery details. While the core protocol naturally resists up to f fully Byzantine faults, Resilient Path clients may additionally assume f_{abc} alive-but-corrupt replicas. These replicas behave correctly during normal operation

Algorithm 2: Fork Detection and Repair for replica r

```

1: upon observing conflicting VoteQCs or a valid  $\langle \text{ALARM}, \text{evidence} \rangle$  do
2:   halt normal operation
3:   broadcast  $\langle \text{ALARM}, \text{evidence} \rangle_r$ 
4:   initiate a Byzantine Broadcast protocol (e.g., Dolev-Strong)
5:   to reliably exchange all locally known CheckpointQCs and VoteQCs

6: upon broadcast protocol completes do
7:    $\mathcal{C} \leftarrow$  all CheckpointQCs delivered by the broadcast
8:   if  $\mathcal{C} \neq \emptyset$  then
9:      $C_{max} \leftarrow$  the  $C \in \mathcal{C}$  with the highest slot  $\triangleright$  Mathematically guaranteed unique per height
10:     $\mathcal{O}_{canon} \leftarrow$  branch of  $C_{max}$  extended per slot with the lowest-digest delivered VoteQC, up to the first gap
11:   else
12:     $\mathcal{O}_{canon} \leftarrow$  the branch built by the same deterministic rule from the initial state
13:   start a new epoch: adopt the final state of  $\mathcal{O}_{canon}$  as the new genesis
14:   resume normal operation from the new genesis

```

but might double-sign conflicting **CHKPROP** messages to intentionally cause forks.

However, because honest replicas only broadcast a single **CHKPROP** per height, the double-signing bound (Lemma 7) explicitly prevents the f_{abc} alive-but-corrupt replicas from successfully forming two conflicting **CheckpointQCs**. Thus, there can be at most one valid **CheckpointQC** per height.

If a **VoteQC** fork occurs (exceeding the core f Byzantine limit), correct replicas immediately detect it via cryptographic evidence, halt normal operation, and force a global repair by broadcasting an **ALARM**. Valid **ALARM** evidence is a pair of **VoteQCs** for the same slot with different digests; since every **CHKPROP** carries its justifying **VoteQC**, a checkpoint split at any height surfaces such a pair. False alarms are impossible in core executions, where conflicting **VoteQCs** would require more than f double-signers. Upon entering this synchronous recovery epoch, every replica initiates its own Byzantine Broadcast instance (such as Dolev-Strong [16]), n in parallel, to reliably exchange all locally known **CheckpointQCs** and **VoteQCs**; correct replicas then operate on the union of the delivered sets. Because Dolev-Strong does not rely on an honest majority, it guarantees that all correct replicas will output the exact same set of valid certificates, even if the total number of faulty replicas ($f + f_{abc}$) constitutes a dishonest majority ($> n/2$). However, if the deployment guarantees that $f + f_{abc}$ remains a strict minority ($< n/2$), this recovery step can be significantly optimized by deploying an honest-majority synchronous broadcast protocol that terminates much faster than Dolev-Strong’s $(f + f_{abc}) + 1$ rounds (e.g., expected constant-round protocols [17] or practical synchronous SMR implementations like Sync HotStuff [15]). When the broadcast protocol completes, replicas collect all delivered **CheckpointQCs** across all slots. At most one **CheckpointQC** exists per height (Lemma 7), and every **CheckpointQC** stored by a correct replica is chain-consistent with all higher ones (Lemma 8); the highest delivered **CheckpointQC** therefore extends every checkpoint a correct replica holds—in particular every client-finalized one. Replicas simply identify the highest **CheckpointQC** and adopt its branch as canonical, extending it deterministically (per slot, the lowest-digest delivered **VoteQC**, up to the first

gap), so all correct replicas derive the same genesis; nothing above C_{max} is resilient-finalized, so any deterministic choice is safe. If no **CheckpointQC** was ever formed, the same rule applies from the initial state. This guarantees that Resilient Path safety remains unbroken.

Post-recovery epochs. Recovery concludes by opening a new epoch: the final state of $\mathcal{O}_{canon}$ becomes the agreed genesis, and normal operation restarts with fresh views, slots, and checkpoint heights under an incremented epoch number. All artifacts carry the epoch number; certificates are ordered lexicographically by (epoch, height), and the single-**CHKPROP** lock applies per (epoch, height). Correct replicas never sign artifacts of a closed epoch, so no stale checkpoint of the old epoch can gather the q witnesses required for a **FinalityQC**. Each epoch is thus an independent execution of *Orcaella* whose genesis commits to all previously finalized state, so Theorems 5 and 6 apply per epoch and Resilient Path safety spans epochs (Lemma 8 and Theorem 9). We acknowledge that recovery restores liveness only until the next fork: the replicas that caused it remain in the committee and can fork the new epoch again. Since every fork exposes more than f provable double-signers, we leave to the operator the policy for seeding the new epoch’s committee, e.g., excluding the provably misbehaving parties, as well as the incentive design that further protects clients [18].

IV. SAFETY AND LIVENESS PROOFS

We show that *Orcaella* satisfies safety and liveness under the network and fault model defined in Section II. Throughout, the quorum threshold is $q = n - f - c$ and the view-change accept threshold is $k = 2f + c + 1$; at the minimal committee size $n = 5f + 3c + 1$, $q = 4f + 2c + 1$. We derive both thresholds directly through these proofs, ensuring a rigorous foundation for the optimal $n \geq 5f + 3c + 1$ bound.

Theorem 1 (Tight Bound Necessity). *For Orcaella (Algorithm 1) to be live under f Byzantine and c crash faults while keeping Fast-Path commits safe across view changes, $n \geq 5f + 3c + 1$ is necessary. At $n = 5f + 3c + 1$, the thresholds are forced: $q = 4f + 2c + 1 = n - f - c$ and $k = 2f + c + 1$.*

Proof. We derive three counting constraints; each is necessary against an explicit adversarial strategy. Throughout, $\mathcal{V}$ denotes the q VIEWCHANGE messages collected by the new leader (Algorithm 1).

Liveness (L): $q \leq n - f - c$. Byzantine replicas may remain silent and crash-faulty replicas may stop at any time, so only $n - f - c$ replicas are guaranteed to respond. Both VoteQC formation and the collection of q VIEWCHANGE messages must complete without faulty participation, hence $q \leq n - f - c$.

Reachability (R): $k \leq 2q - n - f$. Suppose a client finalizes h for slot s via a VoteQC with vote quorum Q_{vote} , $|Q_{vote}| = q$. The adversary crashes the (up to c) crash-faulty members of Q_{vote} before the view change and schedules delivery so that $\mathcal{V}$ contains as few members of Q_{vote} as possible. Since $|\mathcal{V}| = q$ and only $n - q$ replicas lie outside Q_{vote} , $\mathcal{V}$ contains at least $2q - n$ members of Q_{vote} , of which up to f are Byzantine and may misreport; only $2q - n - f$ reports for h are thus guaranteed. If $k > 2q - n - f$, the adversary makes slot s miss the threshold, so even an honest new leader sets $\mathcal{O}[s] = \perp$ and proposes a fresh block B' with $H(B') \neq h$. Non-equivocating replicas, who have not yet voted in the new view, vote for B' , forming a conflicting VoteQC and violating safety. Hence $k \leq 2q - n - f$.

Exclusivity (E): $k \geq n - q + f + 1$. While a VoteQC for h exists, at least $q - f$ non-equivocating replicas voted for h . Since each VIEWCHANGE message carries a single highest-view vote per slot, each non-equivocating replica supports at most one digest for s : at most $(n - f) - (q - f) = n - q$ non-equivocating replicas can report a conflicting digest h' , joined by up to f Byzantine misreports, so any $\mathcal{V}$ contains at most $n - q + f$ reports for h' . This bound is realizable: a Byzantine leader of view v equivocates between h and h' ; $q - f$ non-equivocating replicas vote for h (completing the VoteQC together with the f Byzantine), the remaining $n - q$ vote for h' , and the Byzantine replicas report h' during the view change. By (R) the digest h always reaches k supporting reports; if additionally $k \leq n - q + f$, the conflicting digest h' also reaches k , and the Byzantine replicas fabricate a report view higher than any honest re-vote for h , so the highest-view adoption rule selects h' even under an honest leader, overwriting the finalized h . Hence $k \geq n - q + f + 1$.

Combining. (R) and (E) give $n - q + f + 1 \leq k \leq 2q - n - f$, i.e., $3q \geq 2n + 2f + 1$. Substituting (L), $3(n - f - c) \geq 3q \geq 2n + 2f + 1$, hence $n \geq 5f + 3c + 1$.

Forced thresholds. At $n = 5f + 3c + 1$, the bound $3q \geq 2n + 2f + 1 = 12f + 6c + 3$ gives $q \geq 4f + 2c + 1$, while (L) gives $q \leq n - f - c = 4f + 2c + 1$; thus $q = 4f + 2c + 1$. Then (E) yields $k \geq n - q + f + 1 = 2f + c + 1$ and (R) yields $k \leq 2q - n - f = 2f + c + 1$; thus $k = 2f + c + 1$. Sufficiency of these thresholds is established by the remaining lemmas in this section. $\square$

A. Quorum intersection and Fast-Path safety

Lemma 2 (Quorum Intersection). *Let $n \geq 5f + 3c + 1$ and $q = n - f - c$. The intersection of any two quorums of size q contains at least $2f + c + 1$ non-equivocating replicas.*

Proof. The intersection of two sets of size q in a universe of size n has size at least $2q - n = 2(n - f - c) - n = n - 2f - 2c$. Under the client-side assumption for the Fast-Path (Section II), there are at most f equivocating replicas in the system. Thus, the number of non-equivocating replicas in the intersection (which includes correct, crash-faulty, and AbC replicas: in core executions, AbC replicas do not equivocate) is at least $(n - 2f - 2c) - f = n - 3f - 2c \geq 2f + c + 1$, where the last inequality uses $n \geq 5f + 3c + 1$. $\square$

Lemma 3 (Fast-Path Uniqueness). *For any given view v and slot s , there can be at most one valid VoteQC.*

Proof. A VoteQC requires q accept votes from distinct replicas for a specific digest h . Suppose, for the sake of contradiction, that two VoteQCs exist in view v for slot s : one for h and one for h' (with $h \neq h'$). By Lemma 2, their respective quorums intersect in at least $2f + c + 1 \geq f + 1$ non-equivocating replicas. This implies that at least one non-equivocating replica cast an accept vote for both h and h' in the same view v , which strictly violates the definition of a non-equivocating replica. Thus, $h = h'$. $\square$

B. View-Change safety

To guarantee that a finalized Fast-Path commit is never overwritten by a subsequent leader, the View-Change protocol must enforce that any newly proposed branch $\mathcal{O}$ preserves previously committed digests. This is achieved via the view-change accept threshold $k = 2f + c + 1$.

Lemma 4 (View-Change Invariance). *If a valid VoteQC for (v, s, h) is formed, then for any subsequent view $v' > v$, if a non-equivocating replica adopts a branch $\mathcal{O}$ proposed by $\text{LEADER}(v')$, it must be that $\mathcal{O}[s] = h$.*

Proof. We proceed by strong induction on v' , with the following induction hypothesis (IH): for every view w with $v < w < v'$, any branch adopted by a non-equivocating replica in view w satisfies $\mathcal{O}[s] = h$. (For $v' = v + 1$ this range is empty and the argument below applies unchanged.)

Let Q_{vote} be the quorum of q accept votes forming the VoteQC for (v, s, h) , and let $\mathcal{V}$ be the set of q VIEWCHANGE messages collected by $\text{LEADER}(v')$. By Lemma 2, the signers of Q_{vote} and of $\mathcal{V}$ intersect in at least k non-equivocating replicas.

We first claim that the highest-view vote for slot s reported by any non-equivocating member of Q_{vote} certifies h . Such a replica cast exactly one vote for s in view v , namely for h . Consider any view w with $v < w < v'$ in which it also voted for s : by the consistency guard of line 4, it voted only for a proposal matching the branch it adopted in view w , and by the IH that branch satisfies $\mathcal{O}[s] = h$; hence that vote is also for h . Votes in views below v are

dominated by the view- v vote. Its highest-view report for s is therefore (w', h) for some $w' \geq v$, proving the claim. In particular, the at least k non-equivocating members of Q_{vote} whose reports appear in $\mathcal{V}$ all support h , so $h \in \mathcal{K}$.

We now show no conflicting digest enters $\mathcal{K}$. Each VIEWCHANGE message carries a single highest-view vote for s , so each replica supports at most one digest. By the claim, every non-equivocating member of Q_{vote} supports h ; hence at most $(n - f) - (q - f) = n - q$ non-equivocating replicas can support a digest $h' \neq h$, joined by at most f Byzantine (possibly fabricated) reports. In total, h' gathers at most $n - q + f = 2f + c < k$ reports, so $h' \notin \mathcal{K}$.

Thus $\mathcal{K} = \{h\}$ and the view-change rule forces $\mathcal{O}[s] = h$. Since replicas deterministically recompute $\mathcal{K}$ from $\mathcal{V}$ before adopting a branch, a NEWVIEW carrying any other value for $\mathcal{O}[s]$ fails verification and is rejected; hence any branch adopted by a non-equivocating replica in view v' satisfies $\mathcal{O}[s] = h$. $\square$

Theorem 5 (Fast-Path Safety). *If two clients finalize (v, s, h) and (v', s, h') via the Fast-Path, then $h = h'$.*

Proof. If $v = v'$, Lemma 3 guarantees $h = h'$. If $v < v'$, the first finalization implies a VoteQC was formed in view v . By Lemma 4, every branch $\mathcal{O}$ adopted by a non-equivocating replica in view v' satisfies $\mathcal{O}[s] = h$, and by the consistency guard of line 4, such replicas vote in view v' only for a proposal B' with $H(B') = \mathcal{O}[s] = h$. A VoteQC for slot s in view v' contains at least $q - f \geq 1$ votes from non-equivocating replicas; hence it certifies h . Thus, $h = h'$. $\square$

C. Liveness

Theorem 6 (Liveness (Core Execution)). *Under partial synchrony, in executions where at most f replicas equivocate (i.e., no AbC-induced fork epoch), the protocol ensures that correct replicas eventually commit new proposals; Resilient Path certificates are then produced as part of normal progress.*

Proof. After the Global Stabilization Time (GST), message delays are bounded by Δ . By standard view-synchronization arguments [7], correct replicas will eventually enter a common view v with a correct leader. During this view, the correct leader will broadcast a valid NEWVIEW message followed by PROPOSE messages. Because there are at least $n - f - c = q$ correct replicas that remain active, they will all receive the leader's proposals, validate them, and broadcast VOTE messages within the timeout. The leader (and all clients) will collect these q votes to form a VoteQC, achieving Fast-Path finality. Consequently, every honest replica that observes a VoteQC for slot s broadcasts a CHKPROP, and these proposals all carry identical content: any VoteQC for slot s —in any view—certifies the same digest h (Lemmas 3 and 4), honest replicas execute slots in order from the same checkpointed prefix, and APPLY is deterministic, so every honest CHKPROP for height s equals (s, h, σ_s) . Since checkpoint artifacts

carry no view tag, the $n - f - c \geq q$ matching proposals aggregate into a CheckpointQC regardless of which view's VoteQC each replica observed first. The $\sigma = \text{APPLY}(h)$ check then passes at every honest replica, so q matching CHKWITNESS messages are collected and a FinalityQC is output. Since we are in the core execution regime (at most f equivocators), no VoteQC forks occur; therefore progress continues indefinitely. If this regime is violated, replicas enter fork recovery, and only Resilient Path safety is claimed. $\square$

D. Resilient Path and fork recovery safety

The Resilient Path provides an extended safety guarantee against an adversary capable of breaking the core f Byzantine limit by leveraging up to f_{abc} alive-but-corrupt replicas.

Lemma 7 (Unique CheckpointQC per Height). *If $f_{\text{abc}} < n - 3f - 2c$, no two conflicting CheckpointQCs can be formed for the same slot height s .*

Proof. By the protocol (Step 2, line 8), an honest replica broadcasts at most one CHKPROP per slot height s . Let C_1 and C_2 be two CheckpointQCs for different state roots at height s . Each requires $q = n - f - c$ distinct signatures. Because honest replicas do not double-sign CHKPROPs at the same height, the intersection between the signers of C_1 and C_2 consists entirely of Byzantine or AbC replicas. The number of intersecting signers is at least $2q - n$. Thus, for two CheckpointQCs to form, the adversary must control at least $2q - n$ nodes. We require $f + f_{\text{abc}} < 2q - n$.

We know $q = n - f - c$, so $2q - n = 2(n - f - c) - n = n - 2f - 2c$. Given our assumption that $f_{\text{abc}} < n - 3f - 2c$, we have:

$$f + f_{\text{abc}} < f + (n - 3f - 2c) = n - 2f - 2c = 2q - n$$

This mathematically prohibits the adversary from successfully forming two CheckpointQCs, proving that at most one can exist per height. $\square$

Lemma 7 rules out conflicts at a single height. To safely adopt the *highest* checkpoint during recovery, we additionally need checkpoints at different heights to agree on their common prefix. Recall (Section III) that state roots bind history: σ_s commits to σ_{s-1} and the applied digest. We call a CheckpointQC *recorded* if at least one correct replica stores it from the moment it forms; in particular, any CheckpointQC for which some correct replica issued a CHKWITNESS is recorded, since Step 3 (line 12) requires observing the certificate. A CheckpointQC assembled privately by the adversary from honest CHKPROPs need not be recorded and may be orphaned by fork recovery; this affects no client, as Resilient-Path finalization requires a FinalityQC (Theorem 9).

Lemma 8 (Checkpoint Chain Consistency). *If $f_{\text{abc}} < n - 3f - 2c$, then for any recorded CheckpointQC C' at height s' and any CheckpointQC C at height $s > s'$, the*

history committed by C contains the content (h', σ') certified by C' at height s' . Hence all recorded *CheckpointQCs* lie on a single chain.

Proof. The signer sets of C and C' each have size $q = n - f - c$ and intersect in at least $2q - n = n - 2f - 2c$ replicas. Since $f + f_{abc} < n - 2f - 2c$, at least one common signer R is non-equivocating (R may be crash-faulty, but was live when signing both certificates). Non-equivocating replicas execute slots in order on an append-only local history, which changes only when fork recovery adopts the canonical branch (Algorithm 2); within normal operation, Lemma 4 pins any slot with a *VoteQC* to its digest across views, and R halts upon detecting a fork. Between recovery epochs, therefore, R 's history at height s' is fixed at the content (h', σ') it checkpoint-proposed when signing C' . Across recovery epochs the content is preserved because C' is recorded: its correct holder inputs it to the Byzantine broadcast, so C' is delivered in every recovery epoch after its formation. By strong induction on the number of recovery epochs preceding the formation of C , the highest delivered *CheckpointQC* C_{max} of each such epoch satisfies $C_{max}.s \geq s'$ and its history contains (h', σ') at height s' (apply the lemma inductively to the pair (C', C_{max}) , whose formation is preceded by strictly fewer recovery epochs); hence the adopted canonical branch—and every honest history thereafter—retains (h', σ') at height s' . Since R signed C , the root σ certified by C was computed on R 's history and therefore commits (h', σ') at height s' . By collision resistance of the state commitment, every history consistent with C agrees with C' at height s' . $\square$

Theorem 9 (Resilient Path Safety). *If clients follow the Resilient Path (finalizing only upon seeing a *FinalityQC*), the states they finalize form a single prefix-consistent history—no two clients commit conflicting states at any height—even across view changes, synchronous recoveries, and post-recovery epochs, provided $f_{abc} < n - 3f - 2c$.*

Proof. A client finalizes a state only if it observes a *FinalityQC*, which requires an underlying valid *CheckpointQC*. By Lemma 7, all *CheckpointQCs* at height s certify the same content (h, σ) . Since honest replicas only broadcast a *CHKWITNESS* matching this uniquely determined checkpoint content (and only if its payload executes to the correct state), no *FinalityQC* can be formed for a conflicting state. Moreover, every checkpoint a client finalizes is recorded: a *FinalityQC* carries q *CHKWITNESS* signatures, of which at least $q - f - f_{abc} - c \geq f + 1$ belong to correct replicas, and a correct replica issues a *CHKWITNESS* only after observing—and storing—the underlying *CheckpointQC* (Step 3, line 12).

During Fork Recovery (Algorithm 2), replicas exchange all known *CheckpointQCs* via a Byzantine Broadcast protocol (e.g., Dolev-Strong). Because Dolev-Strong does not rely on an honest majority, it guarantees identical outputs for all correct nodes despite a potentially dishonest majority ($f + f_{abc} > n/2$). Alternatively, if $f + f_{abc} < n/2$, an

honest-majority broadcast can be deployed to terminate much faster (e.g., in expected constant rounds [17], [15]). In either case, all correct replicas will obtain the exact same set $\mathcal{C}$ of *CheckpointQCs*, and $\mathcal{C}$ contains every recorded *CheckpointQC*—in particular every client-finalized one. By Lemma 7 at most one *CheckpointQC* exists per height, and by Lemma 8 the *CheckpointQC* with the highest slot C_{max} extends the content certified by every recorded *CheckpointQC* at lower heights. All correct replicas deterministically adopt the branch of C_{max} , which therefore preserves any state finalized by a Resilient Path client prior to the fork. $\square$

V. ORCDAG: A DAG-BASED IMPLEMENTATION

Sections II and IV are independent of how proposals are represented: safety uses first-round votes and the thresholds q and k of Theorem 1. The Orcaella protocol (Section III) uses q for every certificate round. Many high-throughput systems nonetheless realize the same logic over a *directed acyclic graph* of blocks or vertices, with edges for dependencies and leaders extending a frontier of the DAG [19], [20], [21], [22], [23], [24], [25]. We present *OrcDAG*, a DAG-based instantiation of *Orcaella*.

A DAG vertex (block) plays the role of the primary's proposal for a slot; a replica issues *accept/reject* once its local validity rules (including dependency checks) hold. The key idea of *OrcDAG* is that a vote is not a separate signed message but a causal edge: a round $R + 1$ block votes for a block B precisely when it *supports* B in the sense of *Mysticeti* [21]— B is the first block by its author for round R encountered in a deterministic depth-first traversal of the voter's causal history—and blames B when it does not (the *Vote* and *Blame* of Algorithm 3). Support matters because causal histories merge: a voter's history may contain several equivocating copies of a leader block, but support selects at most one block per author-round slot. Hence a non-equivocating author casts exactly one vote per slot, and every *OrcDAG* execution maps to an execution of *Orcaella*, so the safety results of Section IV apply verbatim. The DAG structure itself carries the first-round votes, so no explicit vote messages are needed even though the DAG is uncertified. The Direct Decision Rule of Algorithm 3 realizes the Fast-Path: collecting q agreeing first-round votes is exactly $\geq q$ distinct round $R + 1$ authors voting for B , using the same quorum $q = n - f - c$ from Theorem 1; symmetrically, q blames force a *Skip*.

The Indirect Decision Rule of Algorithm 3 realizes the View-Change. The “later authenticated artifact” of the abstract protocol *Orcaella* is concretely a committed anchor A at a round $\geq R + 2$: because reachability in the DAG certifies the votes enclosed in its causal past, A linking to $\geq k$ distinct round $R + 1$ authors that vote for B plays the role of the embedded vote reports in the view change of Section III, with the accept threshold $k = 2f + c + 1$. This single anchor object subsumes the abstract alternatives—such as a commit certificate on a descendant, a checkpoint

Algorithm 3: OrcDAG Instantiation (for leader block B at round R)

```

1: Definitions:
   Support: A block  $b'$  supports  $B \equiv (A, R, h)$  if  $B$  is the first block by author  $A$  in round  $R$  encountered in the deterministic depth-first traversal of  $b'$ 's causal history [21].
   Vote: A block in round  $R + 1$  that supports  $B$ .
   Blame: A block in round  $R + 1$  that does not support  $B$ .

2: Direct Decision Rule (evaluated when round  $R + 1$  blocks are delivered):
3: if blocks from  $\geq q$  distinct authors in round  $R + 1$  vote for  $B$  then Commit( $B$ )
4: if blocks from  $\geq q$  distinct authors in round  $R + 1$  blame  $B$  then Skip( $B$ )

5: Indirect Decision Rule (evaluated when anchor  $A$  is committed at round  $\geq R + 2$ ):
6: if  $B$  is undecided then
7:   if  $A$  links to blocks from  $\geq k$  distinct authors in round  $R + 1$  that vote for  $B$  then
8:     Commit( $B$ )
9:   else
10:    Skip( $B$ )

```

quorum, or a bundle hashing prior votes—as all collapse to the anchor’s causal history, which reachability certifies without any separate certificate. The choice of k is exactly what the Reachability, Exclusivity, and Liveness conditions of Section IV require for safe indirect commits.

Appendix A provides detailed algorithms to formally define OrcDAG.

VI. EVALUATION

OrcDAG is a DAG-based instantiation of Orcaella that we implement and benchmark, built in Rust as a fork of Mysticeti [26]. Appendix B details its implementation and testing methodology.

A. Evaluation scope

Our evaluation has a single goal: quantify the latency we gain by trading away fault tolerance, i.e., by moving from the optimal $n = 3f + 1$ three-message-delay fault budget to Orcaella’s $n = 5f + 3c + 1$ two-message-delay design space. Put differently: *what latency do we gain by giving up the fault tolerance represented by the orange wedge of Figure 1 (Section I)?* This is not trivial. A naive observer might expect a direct 33% latency reduction simply from cutting one of the three commit rounds. But two factors complicate the picture: (1) Orcaella needs a larger quorum of replicas per round than a typical $n = 3f + 1$ protocol; and (2) the saved round only reduces the latency from block proposal to commit, not *queuing latency* (from transaction submission to block proposal).

Our evaluation makes the following claims:

- **C1:** OrcDAG’s lower latency is bought with fault tolerance alone, not throughput.
- **C2:** Under benign crash faults, OrcDAG degrades gracefully, retaining its throughput and low latency.
- **C3:** OrcDAG’s ordering-latency reduction over the $n = 3f + 1$ baseline is $\sim 24\%$ at low load and erodes only mildly (to $\sim 22\%$) at high load.
- **C4:** OrcDAG’s ordering latency is independent of how the fault budget splits between f and c in a typical geo-distribution, and a split budget deploys at strictly smaller n than the pure-Byzantine $5f + 1$ ($c = 0$) point.

- **C5:** Geo-distribution is decisive: OrcDAG’s latency advantage holds when all protocols operate over the same geo-locations, but is negated when the baseline’s smaller quorum can exclude a remote region that OrcDAG’s larger quorum cannot.
- **C6:** The checkpoint engine is virtually overhead-free: enabling it changes neither OrcDAG’s throughput nor its ordering latency.
- **C7:** End-to-end latency is $\sim 2\times$ the ordering latency for OrcDAG and the baseline alike, and OrcDAG’s end-to-end latency is $\sim 20\text{--}25\%$ below the baseline’s across all loads and (f, c) splits.

Benchmarking BFT protocols under actual Byzantine behavior is an open problem [27], [28]; the state of the art establishes worst-case guarantees through formal proofs, which we give in Section IV.

B. Experimental setup

To demonstrate these claims, we deploy four OrcDAG configurations spanning the fault-budget spectrum (Figure 2): *Byzantine-only* ($f=10, c=0$), *Byzantine-heavy* ($f=8, c=3$), *Balanced* ($f=6, c=6$), and *Crash-heavy* ($f=2, c=13$). We use two baselines: (i) Mysticeti [21], the closest $n = 3f + 1$ protocol to OrcDAG in terms of both design and implementation, representing the standard 3-round PBFT-style baseline; (ii) Hydrangea [3], an alternative hybrid (f, c) design with $n = 3f + 2c + k + 1$ that can commit in a single round-trip (via an optimistic path) and is, to our knowledge, the only other protocol in this design space with a deployable implementation. The pure-Byzantine $c = 0$ point (an $n = 5f + 1$ deployment) is itself an OrcDAG configuration, which we include as an additional reference rather than as a separate system.

Several recent protocols explore designs in the same neighborhood (Section VII), including Minimmit [2], Kudzu [4], and Alpenglow [29]. None of these provide deployable implementations with networking code; their codebases are intended for simulation only and are therefore excluded from our WAN measurements. The remaining systems, Mysticeti and Hydrangea, both have mature deployable implementations and serve as our baselines.

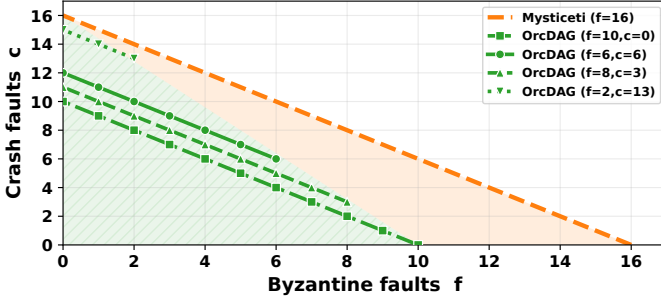


Fig. 2: Runtime fault tolerance of the benchmarked configurations.

We configure Hydrangea at $f=9$, $c=10$, $k=2$ for the $n=50$ committee; in this regime it tolerates more crash faults than OrcDAG, but, being built atop HotStuff, it inherits the data-dissemination bottleneck identified by Narwhal [19].

Unless stated otherwise, we emulate a typical blockchain replica distribution [30], [31]: a fast quorum spanning several EU and US regions, plus a remote tail (Tokyo). We report median (p50) latencies with p90 whiskers. Appendix C describes the precise geo-distribution and testbed details used in this section. In all graphs, *ordering latency* refers to the time elapsed from the moment a client submits a transaction to when it is ordered by consensus (including queuing); *checkpoint latency* refers to the additional time until the transaction is covered by a *certified checkpoint*, that is, a commitment to the executed state attested by a quorum (Appendix B); *end-to-end latency* is their sum; and *throughput* refers to the number of (512 bytes) transactions processed per second.

C. Throughput vs. fault tolerance trade-off

Figure 3 evaluates a roughly 50-replica WAN deployment under failure-free conditions. The throughput of OrcDAG matches Mysticeti, which is expected as both build upon the same uncertified-DAG fabric, corroborating related work [19]. For cost reasons, we cap the input load at 100,000 tx/s, which is two orders of magnitude above the peak throughput observed in deployed blockchains and ample to stress the systems [32]. This confirms claim C1: OrcDAG is realizable without sacrificing throughput. Hydrangea performs significantly worse at scale (red line in fig. 3). At 50 replicas, its latency exceeds the plotted range even under minimal load, demonstrating that it does not scale gracefully to large committees. This limitation stems from its leader-driven data-dissemination mechanism, consistent with findings reported in related work [19].

The figure also shows that OrcDAG orders at lower latency than Mysticeti; we dissect this latency–fault-tolerance trade-off in Section VI-E.

D. Impact of benign crash faults

Figure 4 shows OrcDAG, Mysticeti, and Hydrangea under benign crash faults in committees of approximately 10 replicas. The DAG-based systems sustain the offered load with a graceful latency inflation relative to the fault-free

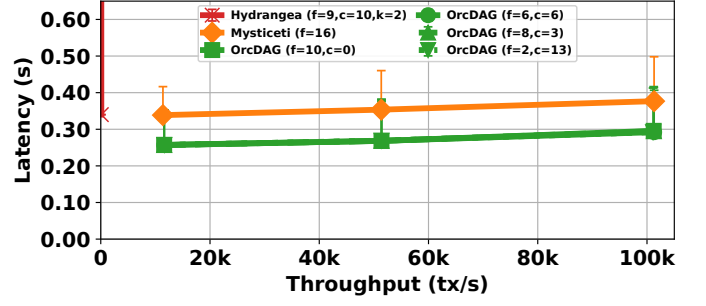


Fig. 3: Throughput vs ordering latency, no faults, 50 replicas.

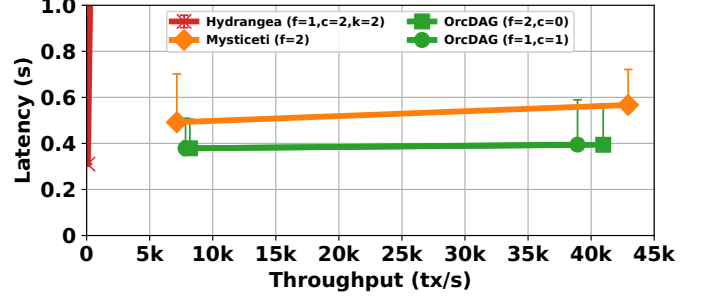


Fig. 4: Throughput vs ordering latency, 2 crash faults, minimal committees.

runs, absorbing crashes by quickly skipping crashed leaders via the direct skip rule (Section V). Each system runs at the minimal committee for this fault budget—Mysticeti at $n=7$ ($f=2$), OrcDAG at $n=9$ ($f=1$, $c=1$), and the $c=0$ reference at $n=11$ ($f=2$)—so every committee operates at its maximum fault load. In these benign fault conditions, OrcDAG matches its own $c=0$ point because crashed replicas count against c rather than f and so do not consume the Byzantine budget (the $n=9$ hybrid and the $n=11$ $c=0$ reference commit within 1 ms of each other: 378 versus 379 ms at low load, 395 versus 394 ms at high load). This split remains latency-neutral while delivering a latency win over Mysticeti: at low load (10,000 tx/s) OrcDAG commits in 378 ms versus 492 ms (a $\sim 23\%$ reduction), and near the high-load cap (50,000 tx/s) 395 ms versus 567 ms (a $\sim 30\%$ reduction). This confirms claim C2. Hydrangea’s latency exceeds the plotted range, confirming it does not tolerate faults gracefully, corroborating related work [19]. These small committees represent the worst case for the fault budget: a crash removes a large fraction of a 10-replica committee but a negligible fraction of a 50-replica one.

E. Understanding the latency trade-off

We zoom into Figure 3 to understand OrcDAG’s ordering-latency–fault-tolerance trade-off. At 100k tx/s, OrcDAG orders in 292–296 ms (p50) versus 377 ms for Mysticeti, a $\sim 22\%$ reduction; at low load (10k tx/s), the gap is 257 ms versus 339 ms, a $\sim 24\%$ reduction. All OrcDAG configurations, including the $c=0$ ($5f+1$) reference, agree within a few milliseconds at both low and high load. A naive view expects the move from three to two message

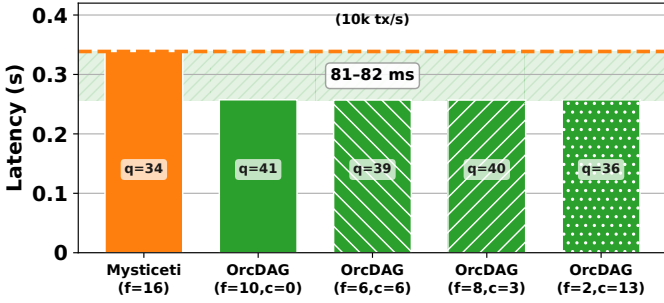


Fig. 5: Per-protocol ordering latency at low load (10k tx/s); load-induced queuing negligible.

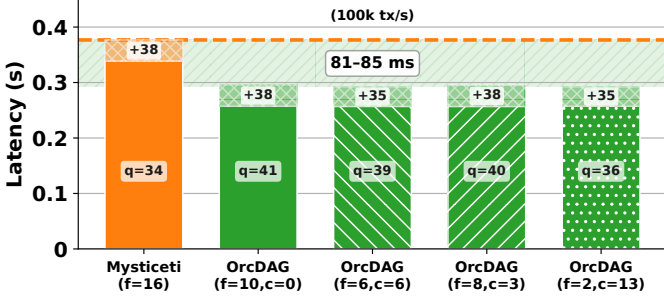


Fig. 6: Per-protocol ordering latency at high load (100k tx/s); load-induced queuing segment stacked.

delays to yield a $\sim 33\%$ reduction in ordering latency. The measured $\sim 24\%$ reduction is explained by the shared, load-independent base: queuing latency’s constant component—since replicas propose a block every round, a transaction waits on average half a round for inclusion—is identical for both systems, meaning the saved round represents a smaller fraction of the total. Moreover, the quorum-width difference ($\sim 80\%$ of replicas for OrcDAG versus $\sim 67\%$ for Mysticeti) costs little in this geo-distribution because both quorums form within the same set of regions (see Section VI-F).

Figure 5 reports per-protocol ordering latency at low load (10k tx/s), which isolates the quorum-width effect because load-induced queuing is negligible. Here, the $\sim 24\%$ reduction holds uniformly across all OrcDAG configurations. Figure 6 adds high load (100k tx/s), introducing load-induced queuing. This steady-state effect (latency stays flat rather than accumulating) arises because larger blocks prolong transmission and higher per-node jitter inflates the wait for the last quorum member. This symmetric queuing adds +38 ms for Mysticeti and +35–39 ms for the OrcDAG configurations. Because additive queuing is common while base ordering latencies differ, the relative reduction erodes mildly, from $\sim 24\%$ to $\sim 22\%$ as load grows from 10k to 100k tx/s, similarly across all (f, c) splits. Figure 7 plots the total ordering-latency reduction over Mysticeti against load; its near load-invariance confirms claim C3.

F. The role of quorum location

Figures 5 and 6 show that, across OrcDAG configurations, ordering latency is not ordered by quorum size. All configurations order in two message delays over the

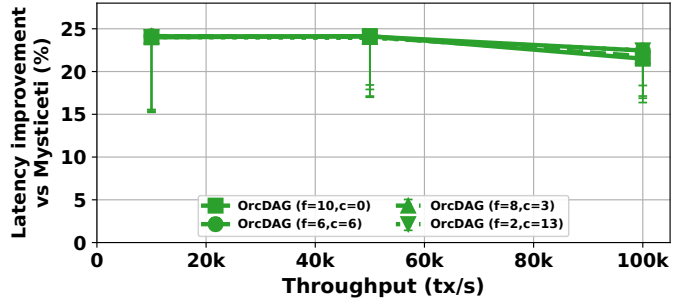


Fig. 7: Ordering latency improvement over Mysticeti vs load.

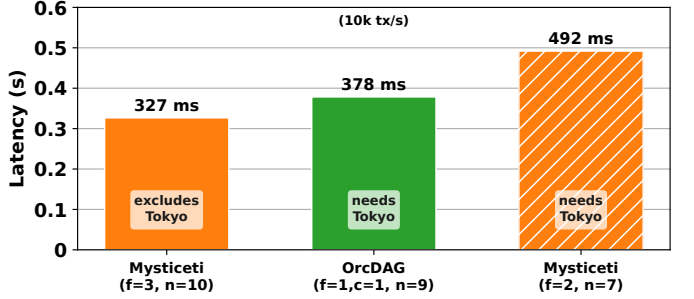


Fig. 8: Latency when the quorum is forced to include the remote tail.

same uncertified-DAG structure, so in a fault-free run their ordering latencies are effectively indistinguishable regardless of how the fault budget splits between f and c . This confirms claim C4: in the common case OrcDAG pays no performance penalty for the hybrid-fault analysis, and a split budget deploys at strictly smaller n than the pure-Byzantine $5f + 1$ ($c = 0$) point. Concretely, the configurations cluster at 257 ms (within 1 ms) at low load and 292–296 ms at high load, versus 339 and 377 ms for Mysticeti; the $c = 0$ (Byzantine-only) point is among the fastest, so the 0–4 ms spread is run-to-run measurement noise (WAN and egress jitter). Crucially, this neutrality is conditional. The split sets the quorum $q = n - f - c$; the Byzantine-leaning configuration carries a larger quorum ($q=41$ for $c=0$) than the crash-leaning one ($q=36$ for Crash-heavy). The configurations are latency-neutral only because, in this graceful geo-distribution, all of these quorums still form in the EU-US without reaching the remote tail in Tokyo.

Figure 8 shows what happens when the quorum is forced to reach beyond the EU-US. We use crash faults on minimal committees (2 crashes) to force the quorum to include the remote tail (Tokyo). When every protocol must reach Tokyo, Mysticeti is penalized more because its extra (third) round waits on the remote replica once more than OrcDAG. But when Mysticeti’s larger fault-tolerance budget gives it enough slack to exclude Tokyo while OrcDAG’s tighter quorum cannot, the advantage reverses. Concretely, Mysticeti with $f=3$ ($n = 10$) keeps slack and excludes Tokyo (327 ms), whereas OrcDAG with $f=1, c=1$ ($n = 9$) and Mysticeti with $f=2$ ($n = 7$) have no slack and must include Tokyo (378 ms and 492 ms). So OrcDAG beats the

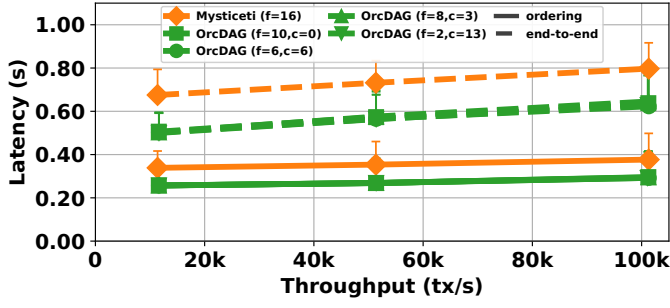


Fig. 9: Ordering (solid) and end-to-end (dashed) latency, no faults, ~ 50 replicas.

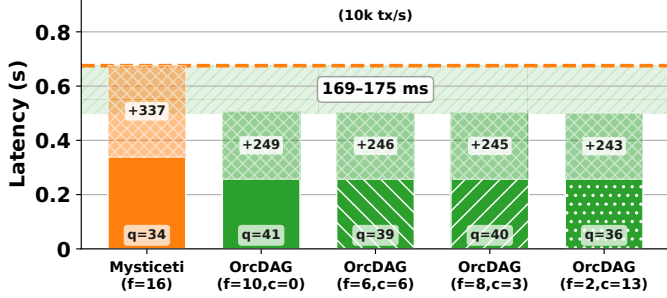


Fig. 10: Latency breakdown at low load (10k tx/s): ordering (solid) plus checkpoint (hatched) latency.

minimal Mysticeti (two rounds versus three) but loses to the over-provisioned Mysticeti that dodges Tokyo. This confirms claim C5. The same mechanism bounds claim C4: under a less graceful geo-distribution, a Byzantine-leaning configuration’s larger quorum ($q=41$) could be forced to include the remote tail while a crash-leaning one ($q=36$) avoids it, at which point the split would no longer be latency-neutral.

G. The cost of checkpoints

We implement the checkpoint engine (Appendix B) emulating the design of production blockchains [33], [34]. It finalizes checkpoints by ordering replica attestations through consensus itself. That is, each replica builds an *attestation*—a small transaction containing a commitment to its executed state—and submits it to the consensus protocol like any other transaction. A checkpoint is *certified* once a quorum of matching attestations is ordered. For OrcDAG, a certified checkpoint carries the Resilient-Path guarantee (FinalityQC, Section III); for the baseline, it provides durability only. We first establish that running the engine affects neither ordering latency nor throughput. As shown in Table II, with the checkpoint engine disabled, OrcDAG ($f=6, c=6$) orders in 253 ms p50 (350 ms p90) at 10k tx/s and 290 ms p50 (404 ms p90) at 100k tx/s. Enabling the engine yields ordering latencies of 257 ms p50 (353 ms p90) and 292 ms p50 (407 ms p90), respectively. The minor 2–4 ms differences are well within standard run-to-run WAN and egress jitter (~ 10 –15 ms). Throughput remains unchanged as both configurations absorb the full offered load. The stream of attestation transactions adds

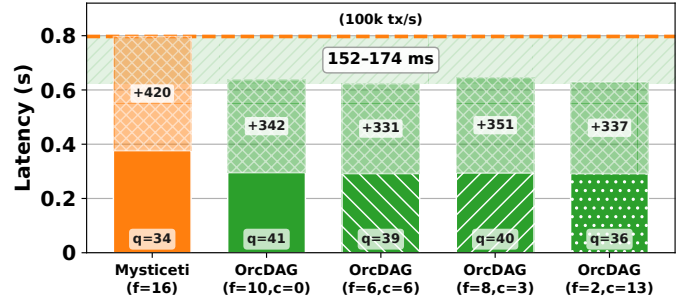


Fig. 11: Latency breakdown at low load (100k tx/s): ordering (solid) plus checkpoint (hatched) latency.

TABLE II: Ordering latency (p50 / p90, ms) with the checkpoint engine enabled and disabled (OrcDAG, $f=6, c=6$).

Load	Engine enabled	Engine disabled
10k tx/s	257 / 353	253 / 350
100k tx/s	292 / 407	290 / 404

only $\sim 1,300$ observations/s (25 checkpoints per second $\times$ 51 replicas), representing a negligible $\sim 1.3\%$ traffic overhead over the main transaction stream at 100k tx/s (and $\sim 14\%$ at 10k tx/s, still with no measurable impact on latency). This confirms claim C6.

As shown in Figure 9, end-to-end latency runs parallel to ordering latency for both systems, costing roughly one additional ordering cycle (end-to-end latency is ~ 2.0 – $2.2\times$ the ordering latency). This is because certification requires a quorum of attestations to be ordered, effectively constituting a second ordering cycle. Two structural factors cause checkpoint latency to slightly exceed a single median ordering cycle: first, a checkpoint is certified only when the q -th attestation is ordered (which tracks a high order statistic of the attestation-ordering distribution rather than the median); second, transactions must wait for the next periodic checkpoint boundary (up to ~ 40 ms). State execution and attestation formation contribute a further negligible 1–8 ms p50. Figures 10 and 11 show that checkpoint latency is 243–249 ms at 10k tx/s and 331–351 ms at 100k tx/s for OrcDAG, compared to 337 ms and 420 ms for Mysticeti. Because OrcDAG saves one message delay in both the ordering of transactions and the ordering of attestations, its latency advantage compounds: its end-to-end latency is 169–175 ms ($\sim 25\%$) below Mysticeti’s at low load, and 152–174 ms ($\sim 20\%$) below at high load. The latency-neutrality of the (f, c) split (claim C4) also extends to end-to-end latency, with all four configurations agreeing within a few milliseconds at all load points. Strikingly, OrcDAG’s end-to-end latency at 100k tx/s (623–645 ms) is lower than Mysticeti’s at 10k tx/s (675 ms). This confirms claim C7. We note that because our implementation certifies checkpoints by ordering attestations through consensus, the four-delay bound of Section III is a lower bound that a DAG-native implementation could approach by certifying attestations once they are causally covered by a quorum of later blocks, rather than waiting for their commit.

VII. RELATED WORK

Hybrid fault models. The formal separation of Byzantine and benign faults was first explored by Thambidurai and Park [35] for interactive consistency. While their work demonstrated that protocols can achieve higher resilience by not treating all failures as worst-case Byzantine, it did not address the latency limitations of state machine replication (SMR). Later pragmatic systems like UpRight [36] and XFT [37] applied this separation to cluster architectures. However, UpRight focuses on end-to-end service robustness rather than theoretical latency minimums, and XFT opportunistically tolerates Byzantine faults only when an honest majority communicates synchronously. In contrast, our work fundamentally re-examines the quorum intersections of SMR under partial synchrony, specifically optimizing for the minimal 2-message-delay commit path while treating f and c as distinct variables.

Low-latency SMR and $5f+1$ protocols. The demand for ultra-low latency in decentralized networks has driven recent protocols to instantiate optimal two-round decision paths. Minimmit [2] relies on a pure counting model that conservatively requires $n \geq 5f + 1$, treating any offline replica as a potential equivocator. Alpenglw [29] refines this regime with a separate allowance for offline stake (its “20+20” model), but its safety holds only when leaders do not equivocate, so it does not withstand the unrestricted Byzantine behavior we consider. Kudzu [4] integrates an optimistic fast path: with $n = 3f + 2p + 1$ replicas, it commits in two rounds only when at most p replicas actually misbehave, reverting to a slower three-round path otherwise. Hydrangea [3] brings this optimistic-fast-path design to a hybrid (f, c) analysis requiring $n = 3f + 2c + k + 1$: its two-round commit is optimistic in the same sense—the fast path remains live only while the number of actual faults is at most a threshold p , beyond which the protocol falls back to a slower, PBFT-style 3-round path. Pushed to its extreme configuration ($k = 2f + c - 3$, hence $n = 5f + 3c - 2$), this optimistic threshold shrinks to $p = f + c - 2$, strictly below the full fault budget $f + c$ the system is configured to tolerate. Hydrangea’s smaller committee therefore does not contradict our lower bound: it dips below $5f + 3c + 1$ only by conditioning fast-path liveness on fewer actual faults than the configured budget. Furthermore, when Hydrangea fails to commit on a leader, it requires at least five message delays to recover and commit that leader’s transactions, whereas Orcaella can complete this in only four. Additionally, unlike Orcaella, Hydrangea lacks a mechanism to rapidly skip unresponsive leaders in the style of Mysticeti [21], making its recovery potentially slower still. The two-step lineage dates to FaB Paxos [8], which established $n \geq 5f + 1$ with a matching lower bound for certificate-forwarding recovery; a theoretical construction can shave this to $5f - 1$ [38], at a complexity cost that has kept practical systems at $5f + 1$. Unlike these approaches, we prove that $n = 5f + 3c + 1$ is necessary for any

protocol that commits in two message delays from a single vote quorum and remains live under the full budget of f Byzantine and c crash faults (Theorem 1), and sufficient via Orcaella—the three extra replicas over Hydrangea’s extreme point buy an unconditional 2-message-delay commit that needs no fallback path, while separating the cost of crash faults from Byzantine faults. Our choice allows us to also provide a resilient path for clients to tolerate an additional f_{abc} equivocators at the cost of two extra message delays, without degrading the core protocol’s optimal liveness in executions where AbC replicas do not act, but results in lower fast-path liveness guarantees than Hydrangea.

Client-side safety and Alive-but-Corrupt faults.

FlexibleBFT [5] introduced the concept of *alive-but-corrupt* (f_{abc}) faults, cleanly separating replica-quorum liveness assumptions from client-visible safety guarantees. While FlexibleBFT primarily uses this slack to separate synchronous and asynchronous network assumptions, we apply it directly to the 2-delay quorum framework. Unlike existing systems that force all clients to accept the same latency-security trade-off, our dual-path architecture explicitly empowers clients. By chaining CheckpointQCs, our Resilient Path allows clients to tolerate an additional f_{abc} equivocators at the cost of two extra message delays; the core protocol’s optimal liveness is unaffected as long as AbC replicas do not act, and if they do, liveness is only temporarily lost until synchronous recovery completes while resilient-path safety is preserved throughout. Orthogonally, a line of low-latency payment systems—FastPay [39], Zef [40], and Stingray [41]—forgoes consensus for single-owner transactions, trading general programmability for latency; Orcaella instead retains full SMR..

VIII. CONCLUSION

We explored the fundamental limits of achieving optimal 2-message-delay consensus under a hybrid fault model. By cleanly separating Byzantine faults (f) from crash faults (c), we derived tight quorum intersections requiring $n \geq 5f + 3c + 1$ with an optimal Fast-Path threshold of $q = n - f - c$ ($4f + 2c + 1$ at the minimal committee size). This allows modern decentralized systems to relax the severe liveness requirements associated with pure $5f + 1$ protocols without sacrificing latency. Building upon these thresholds, we introduced Orcaella, a dual-path consensus protocol that explicitly exposes a latency-safety trade-off to clients. Clients requiring ultra-low latency can finalize in two message delays via the Fast-Path (relying on VoteQCs), while clients prioritizing safety can wait four message delays for the Resilient Path (chaining CheckpointQCs and FinalityQCs). By enforcing a strict single-proposal rule for checkpoints, the Resilient Path guarantees safety against an extended set of f_{abc} alive-but-corrupt replicas. If the core f threshold is ever breached, the system safely halts and employs an Authenticated Byzantine Broadcast recovery mechanism to deterministically salvage the Resilient Path.

REFERENCES

- [1] M. Castro and B. Liskov, “Practical byzantine fault tolerance,” in *Proc. OSDI*, 1999.
- [2] B. K. Chou, A. Lewis-Pye, and P. O’Grady, “Minimmit: Fast finality with even faster blocks,” arXiv:2508.10862, 2025.
- [3] N. Shrestha, A. Kate, and K. Nayak, “Hydrangea: Optimistic two-round partial synchrony with improved fault resilience,” Cryptology ePrint Archive, Report 2025/1112, 2025.
- [4] V. Shoup, J. Sliwinski, and Y. Vonlanthen, “Kudzu: Fast and simple high-throughput BFT,” in *DISC*, 2025.
- [5] D. Malkhi, K. Nayak, and L. Ren, “Flexible byzantine fault tolerance,” in *Proc. ACM CCS*, 2019.
- [6] S. Bano, A. Sonnino, M. Al-Bassam, S. Azouvi, P. McCorry, S. Meiklejohn, and G. Danezis, “SoK: Consensus in the age of blockchains,” in *ACM AFT*, 2019.
- [7] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *J. ACM*, vol. 35, no. 2, pp. 288–323, 1988.
- [8] J.-P. Martin and L. Alvisi, “Fast Byzantine consensus,” *IEEE Transactions on Dependable and Secure Computing*, vol. 3, no. 3, pp. 202–215, 2006.
- [9] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: A tutorial,” *ACM Computing Surveys*, vol. 22, no. 4, 1990.
- [10] S. Cohen, R. Gelashvili, E. Kokoris-Kogias, Z. Li, D. Malkhi, A. Sonnino, and A. Spiegelman, “Be aware of your leaders,” in *Financial Cryptography*, 2022.
- [11] B. Y. Chan and E. Shi, “Streamlet: Textbook streamlined blockchains,” in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 2020, pp. 1–11.
- [12] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “HotStuff: BFT consensus with linearity and responsiveness,” in *PODC*, 2019.
- [13] R. Gelashvili, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman, and Z. Xiang, “Jolteon and Ditto: Network-adaptive efficient consensus with asynchronous fallback,” in *Financial Cryptography and Data Security (FC)*, 2022.
- [14] T. Kichidis, L. Kokoris-Kogias, A. Koshy, I. Sergey, A. Sonnino, M. Tian, and J. Zhang, “Beluga: Block synchronization for BFT consensus protocols,” <https://arxiv.org/abs/2511.15517>, 2025.
- [15] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, “Sync hotstuff: Simple and practical synchronous state machine replication,” in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 106–118.
- [16] D. Dolev and H. R. Strong, “Authenticated algorithms for byzantine agreement,” *SIAM Journal on Computing*, vol. 12, no. 4, pp. 656–666, 1983.
- [17] J. Katz and C.-Y. Koo, “On expected constant-round protocols for byzantine agreement,” in *Advances in Cryptology—CRYPTO 2006*. Springer, 2006, pp. 445–462.
- [18] Z. Avarikioti, E. Kokoris-Kogias, R. Neiheiser, and C. Stefo, “CoBRA: A universal strategyproof confirmation protocol for quorum-based proof-of-stake blockchains,” 2025.
- [19] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus,” in *EuroSys*, 2022.
- [20] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in *ACM CCS*, 2022.
- [21] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, “Mysticeti: Reaching the latency limits with uncertified DAGs,” in *NDSS*, 2025.
- [22] P. Jovanovic, L. Kokoris-Kogias, B. Kumara, A. Sonnino, P. Tennaage, and I. Zablitchi, “Mahi-Mahi: Low-latency asynchronous BFT DAG-based consensus,” 2024.
- [23] N. Shrestha, A. Kate, and K. Nayak, “Sailfish: Towards improving the latency of DAG-based BFT,” in *IEEE Symposium on Security and Privacy (S&P)*, 2025.
- [24] A. Spiegelman, B. Arun, R. Gelashvili, and Z. Li, “Shoal: Improving DAG-BFT latency and robustness,” in *Financial Cryptography and Data Security (FC)*, 2024.
- [25] G. Tsimos, A. Kichidis, A. Sonnino, and L. Kokoris-Kogias, “HammerHead: Leader reputation for dynamic scheduling,” in *IEEE ICDCS*, 2024.
- [26] M. Labs, “Mysticeti: Low-latency DAG consensus with fast commit path,” <https://github.com/asonnino/mysticeti>, 2024.
- [27] S. Bano, A. Sonnino, A. Chursin, D. Perelman, Z. Li, A. Ching, and D. Malkhi, “Twins: BFT systems made robust,” in *Proc. International Conference on Principles of Distributed Systems (OPODIS)*, 2021.
- [28] G. Giuliani, A. Sonnino, M. Frei, F. Streun, L. Kokoris-Kogias, and A. Perrig, “An empirical study of consensus protocols’ DoS resilience,” in *ACM AsiaCCS*, 2024.
- [29] Q. Knip, K. Sliwinski, and R. Wattenhofer, “Solana Alpenglow consensus: Increased bandwidth, reduced latency,” Anza white paper, v1.1, 2025.
- [30] Helius, “Solana decentralization: Facts and figures,” <https://www.helius.dev/blog/solana-decentralization-facts-and-figures>, 2024, accessed June 2026.
- [31] Suiscan, “Suiscan: Sui mainnet explorer,” <https://suiscan.xyz/mainnet/home>, accessed June 2026.
- [32] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A blockchain combining broadcast and consensus,” in *ACM CCS*, 2024.
- [33] M. Labs, “Sui: A next-generation smart contract platform,” <https://github.com/MystenLabs/sui>, 2024.
- [34] A. Labs, “Aptos: A layer-1 blockchain for widespread adoption,” <https://github.com/aptos-labs/aptos-core>, 2024.
- [35] P. Thambidurai and Y.-K. Park, “Interactive consistency with multiple failure modes,” in *Proceedings of the 7th Symposium on Reliable Distributed Systems*. IEEE, 1988, pp. 93–100.
- [36] A. Clement, M. Kapritsos, S. Lee, Y. Wang, L. Alvisi, M. Dahlin, and T. Riche, “Upright cluster services,” in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, 2009, pp. 277–290.
- [37] S. Liu, P. Viotti, C. Cachin, V. Quema, and M. Vukolic, “XFT: Practical fault tolerance beyond crashes,” in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 485–500.
- [38] P. Kuznetsov, A. Tonkikh, and Y. X. Zhang, “Revisiting optimal resilience of fast Byzantine consensus,” in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC)*, 2021, pp. 343–353.
- [39] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-performance byzantine fault tolerant settlement,” in *ACM AFT*, 2020.
- [40] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, scalable, private payments,” in *WPES@CCS*, 2023.
- [41] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast concurrent transactions without consensus,” <https://arxiv.org/abs/2501.06531>, 2025.
- [42] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is DAG,” in *PODC*, 2021.
- [43] T. T. Team, “Tokio,” <https://tokio.rs>, 2024.
- [44] H. de Valence, “Ed25519 for consensus-critical contexts,” <https://crates.io/crates/ed25519-consensus>, 2024.
- [45] Z. Li, A. Sonnino, and P. Jovanovic, “Performance of EdDSA and BLS signatures in committee-based consensus,” in *APPLIED@PODC*, 2023.
- [46] RustCrypto, “Rustcrypto: Hashes,” <https://github.com/RustCrypto/hashes>, 2024.
- [47] Die.Net, “writev(3) – linux man page,” <https://linux.die.net/man/3/writev>, 2024.
- [48] Meta, “Sapling (Minibytes),” <https://github.com/facebook/sapling/tree/main/eden/scm/lib/minibytes>, 2024.
- [49] L. Lamport, “The part-time parliament,” *ACM Transactions on Computer Systems (TOCS)*, vol. 16, no. 2, pp. 133–169, 1998.

APPENDIX A

DETAILED ALGORITHMS FOR THE DAG-BASED VARIANT

This appendix complements Section V by formally defining the commit logic of OrcDAG, the DAG-based

Algorithm 4: Decision Rules

```

1: leadersPerRound  $\triangleright$  A number between 1 and  $q$ 
2: waveLength  $\triangleright$  Set to 2 for Orcaella

3: procedure TRYDECIDE( $r_{committed}, r_{highest}$ )
4:    $S \leftarrow []$   $\triangleright$  Holds decisions
5:   for  $r \leftarrow r_{highest}$  down to  $r_{committed} + 1$  do
6:     for  $l \leftarrow \text{leadersPerRound} - 1$  down to 0 do
7:        $i \leftarrow r \bmod \text{waveLength}$ 
8:        $D \leftarrow \text{DECIDER}(i, l)$ 
9:        $w \leftarrow D.\text{WAVELENGTH}(r)$ 
10:       $s \leftarrow D.\text{TRYDIRECTDECIDE}(w)$ 
11:      if  $s = \perp$  then  $s \leftarrow D.\text{TRYINDIRECTDECIDE}(w, S)$ 
12:       $S \leftarrow s \parallel S$ 
13:   return  $S$ 

14: procedure EXTENDCOMMITSEQ( $r_{committed}, r_{highest}$ )
15:    $S \leftarrow \text{TRYDECIDE}(r_{committed}, r_{highest})$ 
16:    $S_{commit} \leftarrow []$   $\triangleright$  Holds committed blocks
17:   for  $s \in S$  do
18:     if  $s = \perp$  then break
19:     if  $s = \text{Commit}(b_{leader})$  then  $S_{commit} \leftarrow S_{commit} \parallel b_{leader}$ 
20:   return LINEARIZESUBDAGS( $S_{commit}$ )  $\triangleright$  Same as DAG-Rider [42]

```

variant of Orcaella, through detailed algorithms.

DAG-building layer. We assume the underlying DAG-building logic of Mysticeti [21]: replicas proceed in logical rounds; in each round every honest replica proposes one block referencing $\geq q$ distinct valid blocks from the previous round; blocks are disseminated to others; only blocks whose entire causal history has been validated are stored locally. The decision logic specified here operates on this local DAG and is independent of how blocks reach the replica.

Entry point and idempotency. Algorithm 4 is the commit-logic entry point. Inline with related work [21], [22], it is idempotent and stateless from the DAG engine's perspective: the engine may invoke it whenever it likes, typically upon receiving and integrating a new block, passing the highest round currently in the local DAG ($r_{highest}$) and the round of the last block already committed ($r_{committed}$). The procedure returns the extension to the commit sequence (possibly empty) that the engine should append to its committed prefix. The entry point is $\text{EXTENDCOMMITSEQ}(r_{committed}, r_{highest})$, which internally calls TRYDECIDE to evaluate each undecided leader slot using the rules in Algorithm 5, then linearises the causal sub-DAG of every newly committed leader (as introduced by DAG-Rider [42]). Algorithm 5 specifies the per-slot decision process and with the supporting helper procedures (GETDECISIONBLOCKS, GETLEADERBLOCKS, LINK).

APPENDIX B IMPLEMENTATION

We implement a networked, multi-core OrcDAG replica in Rust by forking the Mysticeti codebase [26], [32]. Our implementation leverages tokio [43] for asynchronous networking, utilizing raw TCP sockets for communication without relying on any RPC frameworks. For cryptographic operations, we rely on ed25519-consensus [44], [45] for asymmetric cryptography and blake2 [46] for

Algorithm 5: Decider Instance and Helpers

```

1: waveOffset =  $i$   $\triangleright$  The first parameter of the Decider (i)
2: leaderOffset =  $l$   $\triangleright$  The second parameter of the Decider (l)
3: waveLength  $\triangleright$  Set to 2 for Orcaella
4: replicas  $\triangleright$  The set of replicas

5: procedure WAVELENGTH( $r$ )
6:   return  $(r - \text{waveOffset}) / \text{waveLength}$ 

7: procedure PROPOSEROUND( $w$ )
8:   return  $(w * \text{waveLength}) + \text{waveOffset}$ 

9: procedure DECISIONROUND( $w$ )
10:  return  $\text{PROPOSEROUND}(w) + (\text{waveLength} - 1)$ 

11: procedure STRONGLY CERTIFIED LEADER( $w, b_{leader}$ )
12:   $B_{decision} \leftarrow \text{GETDECISIONBLOCKS}(w)$ 
13:  return  $|\{b'.author : b' \in B_{decision} \wedge \text{SUPPORT}(b_{leader}, b')\}| \geq q$ 
    $\triangleright q = n - f - c$ ; count authors, as replicas may equivocate

14: procedure SKIPPED LEADER( $w, b_{leader}$ )
15:   $B_{decision} \leftarrow \text{GETDECISIONBLOCKS}(w)$ 
16:  return  $|\{b'.author : b' \in B_{decision} \wedge \neg \text{SUPPORT}(b_{leader}, b')\}| \geq q$ 
    $\triangleright q = n - f - c$ 

17: procedure TRYDIRECTDECIDE( $w$ )
18:   $B_{leader} \leftarrow \text{GETLEADERBLOCKS}(w, \text{leaderOffset})$ 
19:  for  $b_{leader} \in B_{leader}$  do
20:    if SKIPPED LEADER( $w, b_{leader}$ ) then return Skip
21:    if STRONGLY CERTIFIED LEADER( $w, b_{leader}$ ) then return Commit( $b_{leader}$ )
22:  return  $\perp$ 

23: procedure WEAKLY CERTIFIED LEADER( $b_{anchor}, b_{leader}$ )
24:   $w \leftarrow \text{WAVELENGTH}(b_{leader}.round)$ 
25:   $B_{decision} \leftarrow \text{GETDECISIONBLOCKS}(w)$ 
26:  return  $|\{b.author : b \in B_{decision} \wedge \text{SUPPORT}(b_{leader}, b) \wedge \text{LINK}(b, b_{anchor})\}| \geq k$   $\triangleright k = 2f + c + 1$ ; votes are supports, anchor certification is reachability

27: procedure TRYINDIRECTDECIDE( $w, S$ )
28:   $r_{decision} \leftarrow \text{DECISIONROUND}(w)$ 
29:   $s_{anchor} \leftarrow \text{first } s \in S \text{ s.t. } r_{decision} < s.round \wedge s \neq \text{Skip}$ 
30:  if  $s_{anchor} = \text{Commit}(b_{anchor})$  then
31:     $B_{leader} \leftarrow \text{GETLEADERBLOCKS}(w, \text{leaderOffset})$ 
32:    if  $\exists b_{leader} \in B_{leader} \text{ s.t. WEAKLY CERTIFIED LEADER}(b_{anchor}, b_{leader})$  then return Commit( $b_{leader}$ )
33:  else return Skip
34:  return  $\perp$ 

35: procedure GETDECISIONBLOCKS( $w$ )
36:   $r_{decision} \leftarrow \text{DECISIONROUND}(w)$ 
37:  return  $\text{DAG}[r_{decision}]$ 

38: procedure GETLEADERBLOCKS( $w, rank$ )  $\triangleright$  Replicas may equivocate
39:   $r_{propose} \leftarrow \text{PROPOSEROUND}(w)$ 
40:   $s \leftarrow r_{propose}$ 
41:   $leader \leftarrow \text{replicas}[(s + rank) \bmod |\text{replicas}|]$ 
42:  return  $\{b \in \text{DAG}[r_{propose}] : b.author = leader\}$ 

43: procedure LINK( $b_{old}, b_{new}$ )
44:  return  $\exists$  a sequence of  $m \in \mathbb{N}$  blocks  $b_1, \dots, b_m$  s.t.
    $b_1 = b_{old} \wedge b_m = b_{new} \wedge \forall j \in [2, m] : b_j \in \bigcup_{r \geq 1} \text{DAG}[r] \wedge b_{j-1} \in b_j.parents$ 

45: procedure SUPPORT( $b_{old}, b_{new}$ )
46:  return  $b_{old}$  is the first block by  $b_{old}.author$  in round  $b_{old}.round$  encountered
   in the deterministic depth-first traversal of  $b_{new}$ 's causal history
    $\triangleright$  Mysticeti support [21]; selects at most one block per (author, round), even if the history contains equivocating copies

```

cryptographic hashing. To ensure data persistence and crash recovery, we employ a Write-Ahead Log (WAL). The WAL optimizes I/O operations through vectored writes [47] and efficient memory-mapped file usage with the `minibytes` [48] crate, minimizing data copying and serialization.

Both OrcDAG and the Mysticeti baseline embed an identical checkpoint engine to support the Resilient Path (Section III). After executing a committed sub-DAG, where execution consists of deterministic parallel writes over the state, each replica derives a cryptographic commitment to the resulting state and forms a checkpoint attestation. The engine samples one checkpoint per committed sub-DAG, yielding ~ 25 checkpoints per second (roughly one checkpoint per 40 ms of committed history). Following the practice of production blockchains [33], the engine introduces no custom checkpoint messages, thereby avoiding dedicated `CHKPROP` and `CHKWITNESS` (Algorithm 1) network paths. Instead, it reuses consensus itself as the broadcast layer, submitting checkpoint attestations as small, specialized (*system*) transactions; a checkpoint is certified once a quorum of matching attestations is ordered.

The key correctness point of this design is that ordering the attestation quorum implicitly yields the witness guarantee of the FinalityQC: once the quorum of matching attestations (forming a CheckpointQC) is ordered, every correct replica observes and stores the certified checkpoint as part of the total order, ensuring that the checkpoint is recorded without any dedicated witness round. For OrcDAG, this mechanism realizes Resilient-Path finality; for the Mysticeti baseline ($n = 3f + 1$), the same engine provides durability only, since the fork bound at $3f + 1$ leaves no alive-but-corrupt (AbC) equivocation slack. The cost of this reuse is latency: checkpoint certification completes one full ordering cycle after the transaction itself is ordered (introducing one extra message delay beyond the protocol-level bound). While dedicated checkpoint messages would save that delay, they would introduce new message types with distinct dissemination, synchronization, and recovery paths, which adds significant engineering complexity; we deliberately keep our engine production-faithful.

In addition to regular unit tests, we inherit and use two supplementary testing utilities from the Mysticeti codebase. First, a simulation layer replicates the functionality of the `tokio` runtime and TCP networking; the simulated network reproduces realistic WAN latencies, while the `tokio` runtime simulator employs a discrete-event simulation approach to model the passage of time. Second, a command-line utility (called the *orchestrator*) [19], [20] deploys real-world clusters of OrcDAG replicas on machines distributed across the globe.

We open-source our OrcDAG implementation, along with its simulator and orchestration tools, to ensure

reproducibility of our results¹².

APPENDIX C TESTBED DETAILS

This appendix complements Section VI by detailing the testbed and experimental setup used to evaluate OrcDAG.

We deploy all protocols on AWS, using `m5d.8xlarge` instances across 6 different AWS regions: Northern Virginia (us-east-1), Ohio (us-east-2), Frankfurt (eu-central-1), London (eu-west-2), Paris (eu-west-3), and Tokyo (ap-northeast-1). Replicas are distributed across those regions as equally as possible. Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB memory, and runs Linux Ubuntu server 24.04.

We instantiate several geo-distributed benchmark clients within each replica submitting transactions in an open-loop model at a fixed rate. We experimentally increase the load of transactions sent to the systems, and record the throughput and latency of commits. As a result, all plots in Section VI illustrate the steady-state latency of all systems under low load, as well as the maximal throughput they can provide after which latency grows quickly. Transactions in the benchmarks are arbitrary and contain 512 bytes. We configure OrcDAG and Mysticeti with 2 leaders per round, and all protocols use a leader timeout of 1 second.

All large-committee failure-free results are obtained from a single measurement campaign, using uniform committees of $n=51$ for every OrcDAG configuration and $n=50$ for Mysticeti. For per-run statistics, we restrict each run to its throughput plateau (scrapes sustaining at least 90% of the peak rate) and report the steady-state floor (the minimum p50 and p90 over this plateau) to ensure robustness to warm-up and cool down; throughput is reported as the median sustained rate. Each run lasts ~ 300 s, with metrics sampled every 15 seconds.

APPENDIX D CRASH-ONLY DEPLOYMENT

When configured with $f = 0$ to tolerate only crashes and no Byzantine faults, Orcaella reduces to an optimal crash-fault-tolerant (CFT) protocol [49]. Figure 12 reports the performance of OrcDAG configured with $f = 0$ and $c = 1$ in a minimal $n = 3$ deployment, where the three replicas run in three distinct, nearby European regions—Frankfurt (eu-central-1), Ireland (eu-west-1), and London (eu-west-2)—to model a regional CFT deployment. The plot shows two loads, 1k and 10k tx/s. Each box spans the mean ± 1 standard deviation, and the whiskers denote ± 2 standard deviations. In this lightly-loaded regime the commit latency is load-independent and essentially flat: OrcDAG commits with a mean latency of about 22 ms at 1k tx/s and about 23 ms at 10k tx/s, with a standard deviation of about 7 ms. This is roughly two orders of magnitude below the hundreds

¹ <https://anonymous.4open.science/r/uncertified-dag-fabric>

² <https://anonymous.4open.science/r/blockchain-validator>

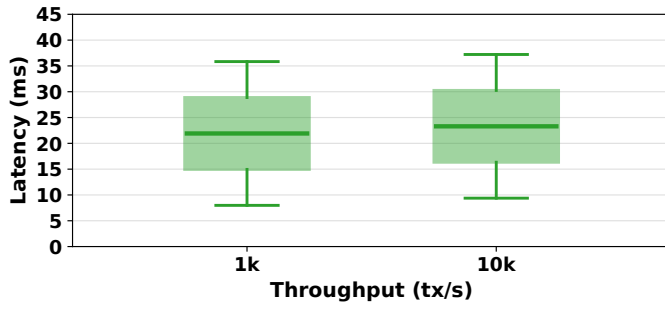


Fig. 12: EU (multi-region) crash-only OrcDAG run; mean $\pm$ stdev.

of milliseconds observed in the geo-distributed WAN runs of the main evaluation (Section VI), highlighting the best-case latency of a regional crash-tolerant deployment.